

Inigo Quilez :: articles :: fast trisect in GLSL - 2020



Intro

There are multiple problems in computer graphics that involve cubic or quartic equations. From [inverting a smoothstep function](#), [computing distances to a parabola](#), [rendering quadratic beziers](#), or [computing intersections with quartic surfaces](#), including that of a [torus](#).

However many developers avoid solving these problems analytically, I believe because creating cubic and quartic equation solvers that are robust is not easy. There are two things one can do that help alleviate this problem:

First, in many cases one can bound the roots easily, say by reducing it to the bbox of the primitive at hand. Often times that's all you need in order to numerically stabilize the solver. Ideally you do it also with quadratic solvers, although it's usually not so critical to do so..

Secondly, you can specialize the solver for the problem at hand. Some geometry problems only accept a single real root and never three, so whole parts of the generic Cardano's solver can be chopped out. Some other times, given the configuration of the shapes, three real roots are guaranteed to exist in all cases, which leads to comparable simplifications. Similarly, often times the coefficients of the cubic come with the numbers {1, 3, 3, 1} premultiplied in them (third row in Pascal's triangle), making many of the intermediate operations in the solver simplify away, which simplifies things further.

Both approaches hint at the same fact -- never use a generic solver, always specialize it to the specific problem at hand. This applies to good old quadratic equations as well. I believe that doing so not only creates a more stable and efficient code, but often also highlights beautiful relationships between the quantities and objects involved. This in turn helps you grow intuitions about the problem, which you otherwise don't when employing a generic solver that obscures the geometric relationships.

Anyway, after all reductions and simplified and optimized, at the center of each cubic usually lays the following operation:

$$y = \cos(\arccos(x)/3)$$

which is reminiscent to me of a Chebyshev polynomial with a fractional index of $1/3$. Anyway, I consider this operation the core of the cubic equation. The operation basically trisects an angle, which is needed to compute the cubic root of unity, which is the heart of the problem. This operation plays the same role than the square root in the resolution of quadratics, which bisects an angle (think of what the square root does to the argument of a complex number).

Assuming we agreed there are cubic problems in computer graphics that we want to solve, it seems natural to wonder if one should give $\cos(\arccos(x)/3)$ a name, and more importantly, should we pursue a hardware implementation of it; or at least a standardized and efficient implementation in software for our shading languages. So, first, let's just call this function "trisect(x)", and quickly move on to the second question, which is interesting and motivated this article - let's think about a direct GLSL implementation.

A Closer Look

We know $\cos(x)$ is one of the fastest operations that you can run in a GPU - it's so fast that any attempt to "optimize" through polynomial approximations or triangle/quadratic replacements turn to be slower always, which often gets newbies and old-school coders alike by surprise.

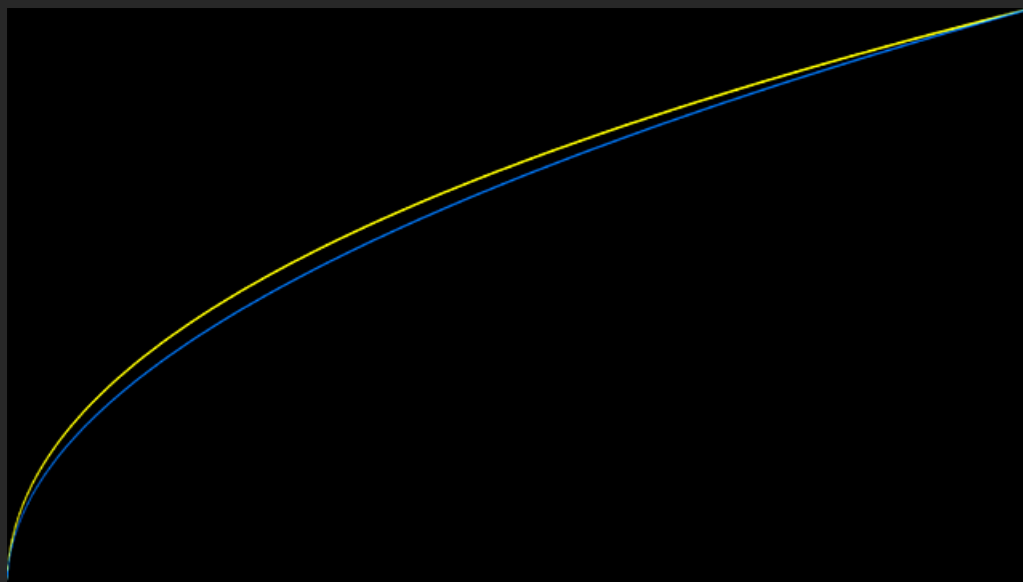
On the other hand, $\arccos(x)$ is a totally different beast. It is usually implemented in software through a polynomial approximation, making it very slow. This function is actually provided for convenience since it is rarely used in rendering tasks, which the GPUs are designed for. As **so many people have said in the past, myself included**, if you encounter an inverse trigonometric function deep in the internals of your renderer, you are probably doing something wrong. And while this is true and therefore GPUs have therefore never had an incentive to have a fast hardware implementation of $\arccos(x)$, it is also a pity.

So, let's put our focus on the software implementation of $\text{trisect}(x) = \cos(\arccos(x)/3)$ again. We are saying $\cos(x)$ has a fast implementation, but $\arccos(x)$ doesn't and it's implemented in software. That got me thinking - since here we are doing an accurate transformation ($\cos(x)$) of an inaccurate approximation ($\arccos(x)$), we might just as well implement the whole $\text{trisect}(x)$ in a single polynomial approximation... maybe?

It turns out we can, with quite positive results.

Manual Play

The first step was to look at the rough shape of the function. The function takes values from -1 to 1, just like $\text{acos}()$, and produces values in the $\frac{1}{2}$ to 1 range. Under a plot, it looks quite like a shifted and biased square root:



Yellow: $\text{trisect}(x)$, Blue: $g_1(x) = \frac{1}{2} + \frac{1}{2} \cdot \sqrt{\frac{1}{2} + \frac{1}{2}x}$

And that was my starting point. I decided to replace the offset and bias in the output of the square root, which is a linear transformation, by a quadratic, cubic and quartic, see if I could bend the curve towards the target $\text{trisect}(x)$. Let's call these quadratic, cubic and quartic approximations $g_2(x)$, $g_3(x)$ and $g_4(x)$, and the direct GLSL implementation of $\text{trisect}(x)$ through $\cos(x)$ and $\text{acos}(x)$, just $f(x)$.

The first thing I did before trying to find the optimal coefficients for these polynomial approximations was to measure their performance with random coefficients. They turned out to run between 2.2 and 1.6 times faster than the reference GLSL implementation $f(x)$. These being great figures, I decided to proceed to find good polynomials and see how accurate results I could get.

Since I wanted to make sure the approximations match $\text{trisect}(x)$ exactly at the domain boundaries I needed $g_i(-1) = \frac{1}{2}$ and $g_i(1) = 1$. That already reduced the search space. In the case of the quadratic it reduced it to a

single parameter, so I could quickly find a good value for it manually in Shadertoy itself, by visually matching $g_2(x)$ to $f(x)$. Actually, the curves matched so well and so easily that it was difficult to say which one was the optimal value for the coefficient. This together with the fact that I wanted to explore the quartic approximation $g_4(x)$ and get it as close to the accuracy of the reference implementation made me take a more serious approach to the exercise.

Parameter Optimization

The first thing I did was to take the reference GLSL implementation $f(x)=\cos(\cos(x)/3)$ and compare its accuracy to a full double precision CPU implementation of $\text{trisection}(x)$. I did this by evaluating $f(x)$ at one million equally spaced points (well, 1024×1024 points), copying the floating point results into a buffer and computing both the maximum absolute error and the MSE (Mean Square Error) relative to the float64 CPU implementation. This gave me

GLSL Reference: Err 1.15e-05, MSE 3.25e-11

Then I modified the code to perform the same measurements on any given of my $g_i(x)$ approximations, starting with a random set of coefficients. I then used vanilla gradient descent for refining the coefficients. I made sure to randomize things enough, and to explore big areas of the parameter space. I am aware there are analytic ways to create the optimal polynomial approximation to a given function, but in this case it was just quicker for me to code that do re-learn Mathematica...

So, after a few hours of gradient descending from randomly chosen starting points, I quickly converged to a set of coefficients with following results:

	Err	MSE	Perf.	Implementation
$f(x)$	1.14e-05	3.26e-11	1.0	1 COS + 1 DIV + 1 ACOS(?)
$g_4(x)$	1.10e-05	5.64e-11	1.6	1 SQRT + 5 FMA
$g_3(x)$	9.65e-05	4.72e-09	1.6	1 SQRT + 5 FMA

$g_2(x)$	1.03e-03	4.15e-07	1.8	1 SQRT + 4 FMA
$g_1(x)$	1.62e-02	1.33e-04	2.2	1 SQRT + 3 FMA
$\text{sqrt}(x)$	n/a	n/a	3.1	1 SQRT

I added $\text{sqrt}(x)$ in the table not because it's a valid approximation, but because it gives us the “Speed of Light” in a universe where $\text{trisect}(x)$ would be implemented in hardware and happened to be as fast as $\text{sqrt}()$ is. The measurements were taken in OpenGL 4.5 and GLSL shaders with `#version 450`. Using precision qualifiers in the shader or optimization pragmas did not change the results.

As we can see, $g_4(x)$ is a pretty good approximation and is already 1.6 times faster than $f(x)$. Even $g_2(x)$ is probably good enough for many applications at almost twice the performance of the GLSL implementation $f(x)$.

The complete implementation (without `fma()` instructions which doesn't exist in WebGL) is live in Shadertoy at <https://www.shadertoy.com/view/WltSD7>, but I copy it here for convenience:

```
float f( in float x )
{
    return cos(acos(x)/3.0);
}

float g1( in float x )
{
    x = sqrt(0.5+0.5*x);
    return x*0.5+0.5;
}

float g2( in float x )
{
    x = sqrt(0.5+0.5*x);
    return x*(-0.064913*x+0.564913)+0.5;
}

float g3( in float x )
{
    x = sqrt(0.5+0.5*x);
    return x*(x*(x*0.021338-0.096562)+0.575223)+0.5;
}

float g4( in float x )
{

```

```
x = sqrt(0.5+0.5*x);  
return x*(x*(x*(x*-0.008978+0.039075)-0.107071)+0.576974)+0.5;  
}
```

Caveats

The coefficients I found are probably not the optimal. However, I feel that even if they were, they might need to be tweaked for each GPU model/architecture out there, since different GPUs implement $\text{sqrt}(x)$, which all of my approximations $g_i(x)$ are based on, to different levels of accuracy. I think $\text{sqrt}()$ is usually computed as $1/\text{inversesqrt}(x)$ or $x*\text{inversesqrt}(x)$, and I think (but I'm not sure) that as a result of it one can expect around only 2ULP precision. If that's so, my gradient descent search surely noticed it and found the best coefficients to work around that fact. However different GPU models with better or worse implementations of $\text{sqrt}(x)$ might need a different set of coefficients. So, if $\text{trisect}(x)$ was to be implemented by the GLSL layer, it'd be great if the driver implementators could provide an implementation of $g_4(x)$ with the best coefficients for any particular GPU model.

inigo quilez - learning computer graphics since 1994